DOCUMENT Q&A; SYSTEM

WITH RAG AND A VECTOR DATABASE

Project Documentation

AI & Machine Learning • Generative AI / NLP / LLM Systems

Project Type: Beginner-friendly end-to-end RAG application

Application: Streamlit web application

LLM: Llama 3.2 via Ollama

Prepared as a practical project report

1. Abstract

This project implements a Document Question Answering System using Retrieval-Augmented Generation (RAG) and a vector database. A PDF document is processed, its text is divided into smaller chunks, and each chunk is converted into a numerical embedding. The embeddings are stored in FAISS, which is used for semantic similarity search. When a user asks a question, the question is embedded and the most relevant document chunk is retrieved. The retrieved context is then passed to the Llama 3.2 language model running locally through Ollama. The generated answer is displayed through a Streamlit web application together with the source document and page number. The system also instructs the model not to invent information when the requested answer is not present in the document.

2. Project Overview

The purpose of the project is to demonstrate how a modern RAG application can answer questions from a private document without relying only on the general knowledge of a language model. The demonstration uses an Employee Leave Policy PDF. The application can answer questions such as the number of annual leave days, medical leave requirements, and leave approval rules.

3. Problem Statement

A general-purpose language model can generate an answer that sounds correct even when the requested information is not contained in a particular document. For document-based question answering, the answer should be grounded in the source material. This project addresses that problem by retrieving relevant document content before asking the language model to generate an answer.

4. Objectives

• Extract text from a PDF document.

• Split the extracted text into smaller chunks.

• Generate semantic embeddings for the chunks.

• Store embeddings in a FAISS vector database.

• Retrieve the most relevant chunk for a natural-language question.

• Use a local LLM to generate an answer from retrieved context.

• Preserve source and page metadata.

• Reduce unsupported or hallucinated answers.

• Provide a simple Streamlit web interface.

• Document the complete workflow so a beginner can reproduce it.

5. Technologies Used

Technology	Purpose
Python	Main programming language
Jupyter Notebook	Development and testing
PyPDF	PDF text extraction
LangChain Text Splitters	Text chunking

Technology	Purpose
Sentence Transformers	Embedding generation
all-MiniLM-L6-v2	Embedding model
FAISS	Vector storage and semantic search
Ollama	Local LLM runtime
Llama 3.2	Language model
Streamlit	Web interface
Pickle	Saving processed document chunks

6. Basic Concepts for a Beginner

6.1 What is RAG?

RAG stands for Retrieval-Augmented Generation. Instead of asking the language model to answer only from its pretrained knowledge, the system first retrieves relevant information from a document or knowledge base. That retrieved information is added to the prompt as context, and the model generates the final answer from that context.

6.2 What are Embeddings?

An embedding is a numerical representation of text. Similar meanings tend to produce vectors that are closer together in embedding space. This allows the system to search for relevant text by meaning rather than by exact keyword matching.

6.3 What is a Vector Database?

A vector database stores numerical embeddings and provides similarity search. In this project, FAISS is used to store the document vectors and find the chunk most similar to a user question.

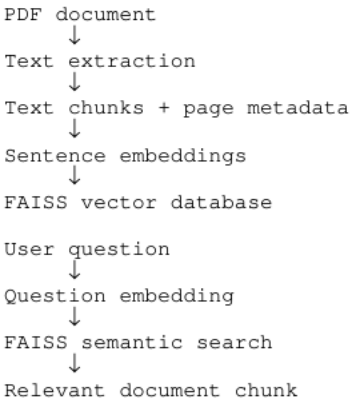
6.4 What is an LLM?

A Large Language Model is a model that can understand and generate natural-language text. This project uses Llama 3.2 through Ollama.

6.5 What is Ollama?

Ollama provides a local way to run supported language models. In this project, the Llama 3.2 model runs on the local computer, and Python sends the RAG prompt to it.

7. System Architecture



```
      ↓
Context + question
      ↓
Llama 3.2 through Ollama
      ↓
Answer + source + page
      ↓
Streamlit interface
```

8. Project Folder Structure

```
project 3/
■■■ app.py
■■■ rag.ipynb
■■■ documents/
■   ■■■ sample.pdf
■■■ vector_db/
    ■■■ document_chunks.pkl
    ■■■ document_faiss_index.bin
```

The notebook contains the development and testing workflow. app.py contains the Streamlit application. The documents folder contains the knowledge-source PDF. The vector_db folder contains the saved document chunks and FAISS index used by the application.

9. Step-by-Step Implementation

9.1 Install the Required Packages

```
pip install pypdf
pip install langchain langchain-community langchain-text-splitters
pip install sentence-transformers
pip install faiss-cpu
pip install streamlit
pip install pandas numpy scikit-learn
pip install reportlab
pip install ollama
```

The packages were installed in the Anaconda environment used to run the project. Installing them in the same environment as Streamlit avoids missing-module errors.

9.2 Create the Document

A sample PDF named sample.pdf was created in the documents folder. It contains an Employee Leave Policy with five main topics: annual leave, leave requests, medical leave, leave approval, and carry-forward leave.

9.3 Extract PDF Text

```
from pypdf import PdfReader

pdf_path = "documents/sample.pdf"
reader = PdfReader(pdf_path)

text = ""
for page in reader.pages:
    page_text = page.extract_text()
    if page_text:
        text += page_text + "\n"
```

The PDF text was successfully extracted and inspected before continuing to chunking.

9.4 Split the Text into Chunks

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=300,
    chunk_overlap=50
)
```

The final metadata-aware processing produced two chunks. Each chunk retained the source file and page metadata.

9.5 Preserve Source and Page Metadata

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("documents/sample.pdf")
documents = loader.load()

document_chunks = text_splitter.split_documents(documents)
```

The metadata included the source path and page information. For the sample PDF, the retrieved information is associated with Page 1.

9.6 Generate Embeddings

```
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
document_texts = [doc.page_content for doc in document_chunks]
document_embeddings = embedding_model.encode(document_texts)
```

The final run produced 2 embeddings with shape (2, 384). This means each of the two chunks was represented by a 384-dimensional vector.

9.7 Create the FAISS Index

```
import faiss

document_index = faiss.IndexFlatL2(document_embeddings.shape[1])
document_index.add(document_embeddings.astype("float32"))
```

The final FAISS index contained 2 vectors. IndexFlatL2 uses L2 distance for similarity search.

9.8 Save the Vector Database Artifacts

```
import pickle
import faiss

with open("vector_db/document_chunks.pkl", "wb") as f:
    pickle.dump(document_chunks, f)

faiss.write_index(
    document_index,
    "vector_db/document_faiss_index.bin"
)
```

Saving the chunks and index allows the Streamlit application to load the processed knowledge base without rebuilding it every time.

9.9 Semantic Search

For a question such as “How many days of annual leave are employees entitled to?”, the question is converted into an embedding. FAISS searches the stored vectors and returns the most relevant document chunk. The retrieved chunk contains the annual leave information and metadata identifying documents/sample.pdf and Page 1.

9.10 Connect Retrieval to the LLM

```
response = ollama.chat(
    model="llama3.2",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ]
)
```

The prompt instructs the model to use only the retrieved context. If the answer is not present, the model is instructed to say that it could not find the answer in the provided document.

9.11 Create the Reusable RAG Function

A reusable ask_rag(question) function was created. It performs the complete sequence: query embedding, FAISS search, retrieval of the document chunk, extraction of context and metadata, prompt construction, Ollama generation, and return of answer, source, and page. This function is then used by the Streamlit application.

9.12 Create the Streamlit Application

The app.py file loads the saved chunks, FAISS index, and embedding model. It provides a question input box and an Ask button. After the user submits a question, the application retrieves context, sends the prompt to Llama 3.2 through Ollama, and displays the answer, source, and page.

```
streamlit run app.py
```

The completed application was successfully opened at the local Streamlit address and tested in a web browser.

10. Final RAG Workflow

```
Question from user
        ↓
Convert question to embedding
        ↓
Search FAISS
        ↓
Retrieve top relevant chunk
        ↓
Extract text + source + page
        ↓
Build grounded prompt
        ↓
Send prompt to Llama 3.2
        ↓
Display answer
        ↓
Display source and page
```

11. Testing and Results

Test question	Observed result	Status
How many days of annual leave are employees entitled to?	12 days of annual leave per year.	PASS
What is required for medical leave?	A valid medical certificate is required.	PASS
Who must approve leave requests?	The manager must approve leave requests.	PASS
What is the employee's monthly salary?	Information not found in the provided document.	PASS

12. Hallucination-Control Test

The question “What is the employee’s monthly salary?” was deliberately selected because salary information does not appear in the PDF. The application returned: “I could not find the answer in the provided document.” This is an important project result because the prompt explicitly limits the model to the retrieved context instead of asking it to use outside knowledge.

13. Source and Page Display

The final application displays the source document and page number with the answer. Example: Source: documents/sample.pdf; Page: 1. This metadata was preserved by using document objects instead of plain strings during the final chunking stage.

14. Advantages

• Semantic search can retrieve relevant text even when the question and source use different wording.

• The answer is grounded in retrieved document context.

• The application displays source and page information.

• The LLM runs locally through Ollama.

• The architecture is simple enough for a beginner to understand and explain.

• The design can be extended to multiple documents.

15. Limitations

• The demonstration currently uses one small PDF.

• Only the top one retrieved chunk is passed to the model.

• The application does not yet provide an upload screen for new documents.

• Formal evaluation on a large question-answer dataset was not performed.

• Answer quality depends on the source document, embedding model, retrieval quality, and LLM.

16. Future Enhancements

• Add PDF upload directly inside Streamlit.

• Support TXT and DOCX files.

• Support multiple documents and document selection.

• Retrieve multiple relevant chunks.

• Add richer citations and document names.

• Add conversation history.

• Evaluate Precision@K, Recall@K, groundedness, and latency on a larger test set.

• Deploy the application for remote access.

17. How to Run the Project

• Open Anaconda Prompt.

• Go to the project folder: C:\Users\DELL UZER\project 3

• Make sure the required packages are installed in the active Anaconda environment.

• Make sure Ollama is installed and the llama3.2 model is available.

• Run: streamlit run app.py

• Open the local Streamlit page in the browser.

18. Conclusion

The project successfully demonstrates a complete document-based RAG question-answering system. The application processes a PDF, creates text chunks and embeddings, stores them in FAISS, retrieves relevant context for user questions, generates grounded answers with Llama 3.2 through Ollama, and displays source and page information through Streamlit.

Testing showed correct answers for questions contained in the document and an appropriate “not found” response for information that was absent. The project therefore provides a practical beginner-level implementation of Generative AI, NLP, semantic search, vector databases, and RAG.